After random MLPs, trained ones: what structure training leaves in a 1024 × 16 net, seen through the cumulant chain (submission #331937)

James Henry (jamesrahenry), with Claude Code

ARC White-Box Estimation Challenge 2026, Phase 2 algorithmic-contribution write-up. 2026-10-28

Submission #331937 (jamesrahenry): adjusted 7.04e-9, final-layer MSE 2.18e-8 at 32 % of budget, 0/100 failures, regraded bit-identical under flopscope 0.12.3 as #337164. It is a clean-room port of ARC's factorized K=3 chain, built from the paper and the reference code without reading any participant's estimator. It reached the V18 band and nothing in it is new. Release and the list of levers we tried and closed: [LINK: estimator release]. This post is about what we did with it afterwards.

The idea

Everything on this board is about He-Gaussian weights, which is the challenge as posed. But the chain is also a very precise null model: it says what a 1024 × 16 ReLU net looks like with zero learned structure. So we trained a few hundred nets at exactly that architecture, kept every initialisation, froze #331937, and pointed it at them to see where training shows up. The corpus (Model Zoo Cartography, MZC) and all the code are public: https://github.com/jamesrahenry/Model_Zoo_Cartography. Everything below was preregistered before the nets were trained, with the prediction files committed first; the hashes are in the repo. Where a prediction failed we say so.

Setup. Bias-free ReLU, width 1024, depth 16, He init, the Phase 2 architecture. Trained as classifiers (Gaussian mixtures with 2 to 50 classes on whitened input; whitened MNIST and Fashion-MNIST at width 512), AdamW, batch 512, learning rate, steps and weight decay varied by grid; about 570 nets over four grids. Each layer splits as $W = \alpha \cdot W0 + \Delta$, the surviving copy of the net's own init plus the trained update. Truth is Monte Carlo at 4 million N(0, I) draws. Two numbers do most of the work: **re-randomised accuracy** (task accuracy after swapping $\alpha \cdot W0$ for a fresh He draw of the same size: how much the function depends on its particular random draw), and the chain's **held-out coherent share** at the last layer (the fraction of its squared error removed by a per-layer scale-and-shift fitted on *other* nets of the same setting: 0 for isotropic error, near 1 for a same-signed shortfall).

What we found

1. Training leaves most of the init in place, by exactly the amount AdamW says, and puts the update where it counts.

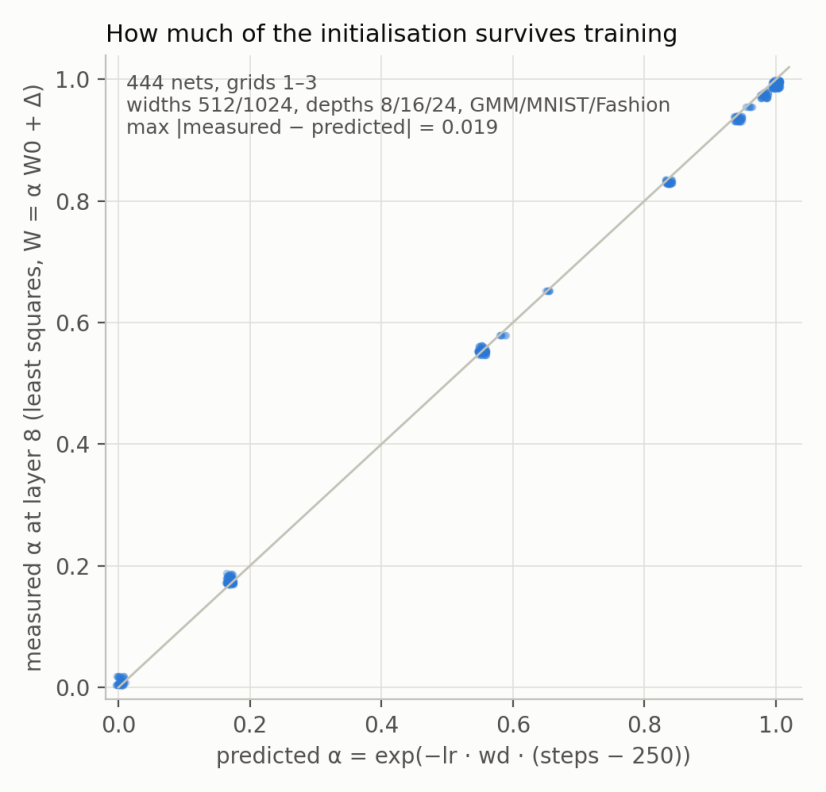


Figure 1: Surviving init amplitude α at layer 8 against exp(−lr·wd·(steps − 250)), 444 nets across widths 512 and 1024, depths 8, 16 and 24, GMM, MNIST and Fashion-MNIST.

Decoupled weight decay shrinks the init copy by exp(−lr·wd·t) and nothing else does: within 0.03 on 108 of 108 nets with decay, $\alpha \geq 0.97$ on all 54 without, worst net 0.019 off. The update Δ is low-rank (participation ratio 8 to 29; half its energy in the top 8 directions per layer). Even at wd 0.3 the random part still carries 60 to 67 % of the weight energy in the middle layers; at wd 0, 91 %. A closure that treats the weights as random is therefore mostly right by energy and wrong where it matters, because the update sits in the signal directions.

2. The function lives in about sixteen directions of the random remnant per layer, the ones that touch the update.

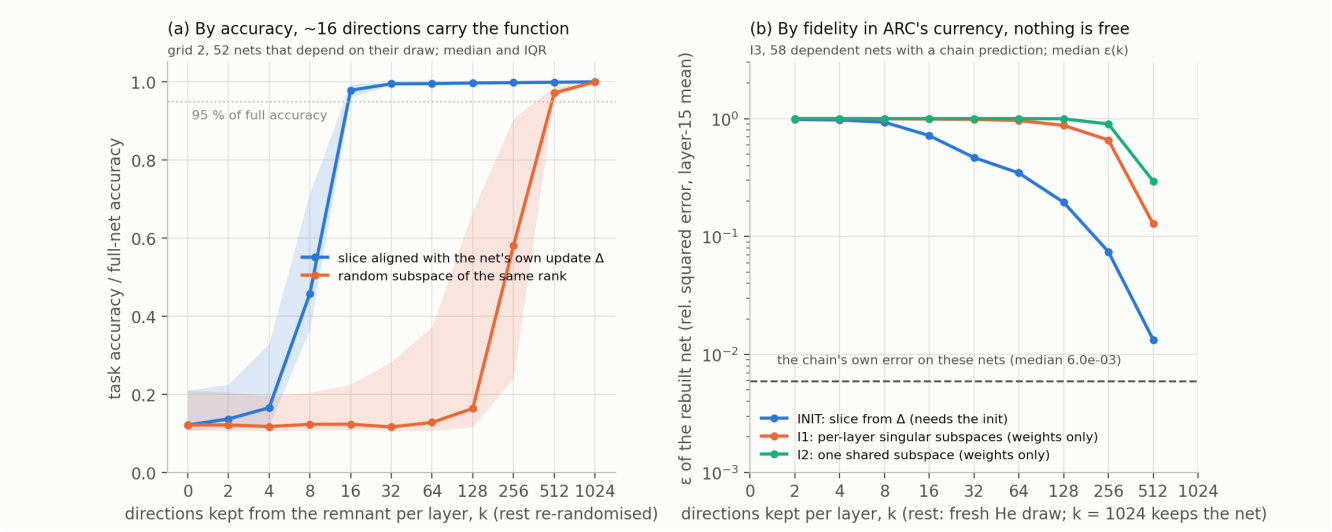


Figure 2: (a) Accuracy against the number of remnant directions kept per layer, for the slice aligned with the net's own Δ and for a random subspace of the same rank (52 draw-dependent nets, median and IQR). (b) The same experiment in ARC's currency: relative squared error of the rebuilt net's layer-15 mean against the chain's own error on those nets (58 nets).

Keep the 16 to 64 directions of α·W0 that touch the input and output subspaces of the net's own Δ, re-randomise the other thousand, and accuracy comes back. A random subspace of the same size needs 4 to 32× more (Fashion-MNIST needed 192 against our preregistered cap of 128; that row is a fail). Judged by ARC's metric instead of accuracy, though, nothing is free: at the 95 %-accuracy rank 97 % of nets have already lost more than the chain's own error, fidelity needs a median 64× the rank that accuracy does, and in high-decay nets whose remnant is "inert" by accuracy, replacing it costs ε = 0.68 against a chain error of 0.013. We also tried to find those sixteen directions from the final weights alone, without the init. Detection works (92 % of nets show cross-layer subspace chaining at ten times chance, their inits do not); recovery does not (no weights-only method reaches the chain's error before keeping essentially everything).

3. Whether a net depends on its random draw and whether the chain gets it wrong are governed by different variables. We guessed wrong about both.

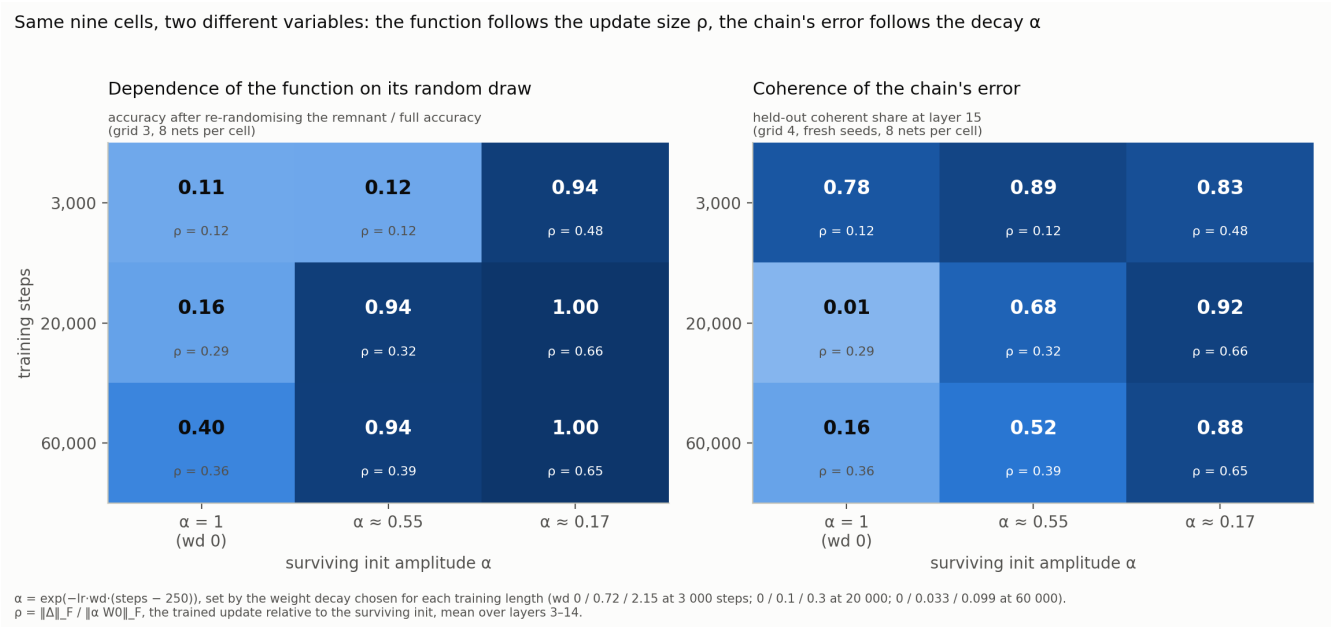


Figure 3: The same nine (training length, α) cells read two ways. Left: how much of the function survives re-randomising the remnant (grid 3, 8 nets per cell). Right: how coherent the chain's error is (grid 4, fresh seeds, 8 nets per cell). The function orders by the update size ρ = ‖Δ‖/‖αW0‖; the chain's error orders by α; the top-left cell breaks both.

We expected weight decay to control everything. Grid 1 (210 nets) agreed, five of five predictions. Grid 2 (162 nets, other widths, depths, real data, other learning rates and lengths) broke it: at the same $\alpha \approx 0.55$ a net trained at lr 1e-4 keeps 0.23 of its accuracy under re-randomisation and one at lr 3e-4 keeps 0.83, and short training leaves nets fully dependent on their draw at any α. Grid 3 (72 nets, decay and training time decoupled) found the relative update size ρ predicts dependence better than α (Spearman 0.90 vs 0.69), but "dependence is a function of ρ alone" failed too: at equal ρ a decayed net is far more independent than an undecayed one.

The chain reads the same cells the other way round. Its coherence tracks decay, not update size (Spearman 0.85 against $-\ln \alpha$, 0.37 against ρ), and every heavily decayed cell is coherent (share ≥ 0.83). But our prediction that undecayed nets give incoherent error at every training length failed: 3 000 steps without decay gives a coherent share of 0.78, a real 5.5e-3 relative error with slope −0.07, while the same recipe at 20 000 steps gives 0.006. We do not have the law for that cell.

4. On trained weights the chain's error is not noise. It is a shortfall along the prediction.

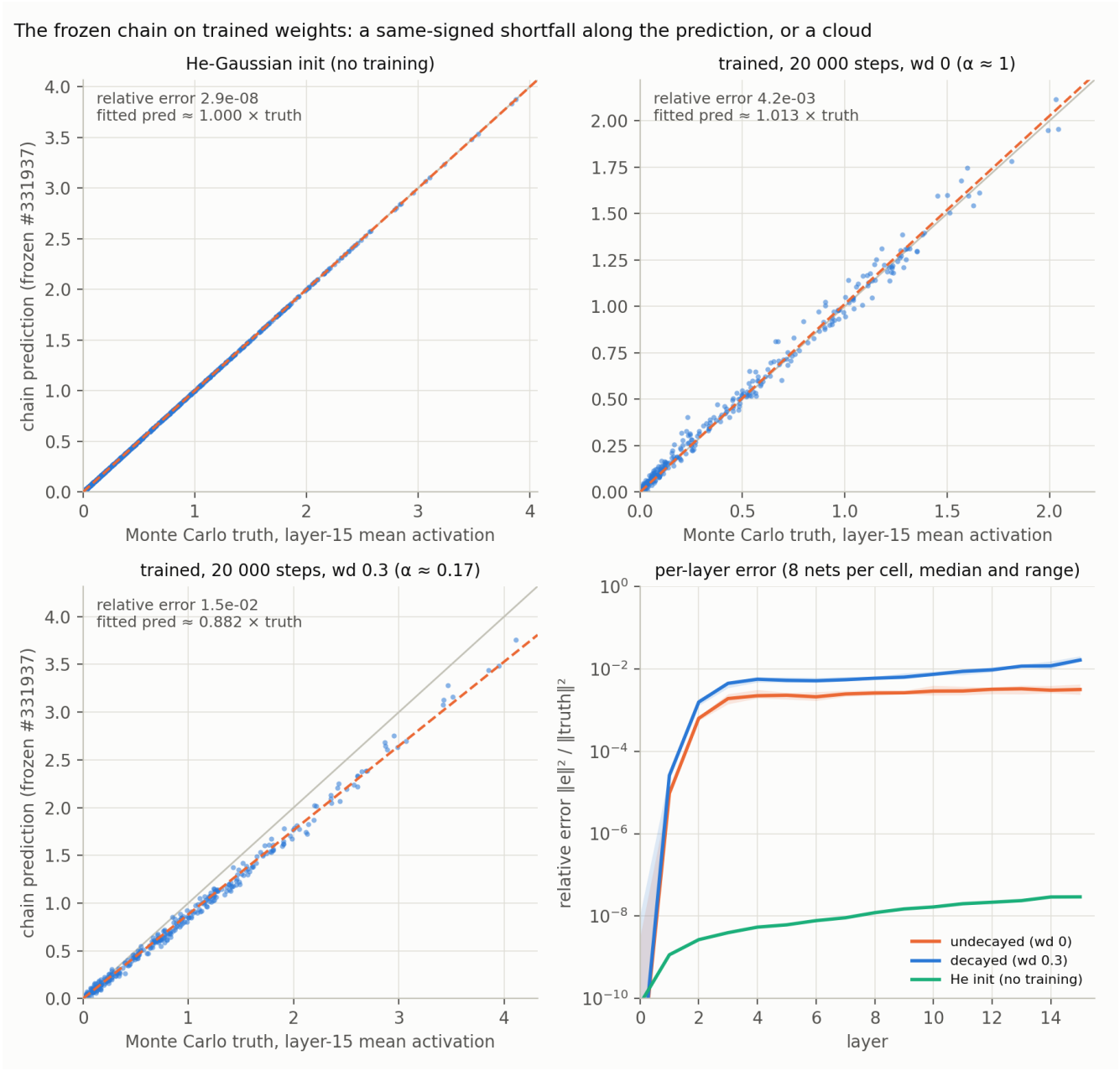


Figure 4: Per-neuron layer-15 mean, Monte Carlo truth against the frozen chain, for a He init, a trained undecayed net and a trained decayed net (same task and length); dashed line is the fitted slope. Bottom right: relative error by layer, 8 nets per cell.

On a fresh He init the chain does what it was built for: relative error 3e-8, points on the diagonal. On a net trained at wd 0.3 the error is five orders larger and almost all one thing: the prediction is about 12 % short of the truth, the same way on every neuron (held-out coherent share 0.88 to 0.92). A per-layer scale-and-shift fitted on *other task families* cuts a held-out family's last-layer MSE to 0.07 to 0.27×, and makes undecayed nets worse. This is the "errors along the mean barely fade" clause of megan_park's survival law (18282; dogus_ozel 18281; AndreasHad04 18292) seen from the trained side: on random weights that direction carries about 1 % of the chain's error, on trained weights nearly all of it.

5. A synthetic "trained" net gets the shape but not the size. Take a shrunken He init (α 0.18), add a rank-16 update whose subspace is shared across layers, rescale to the He fixed point, no data, no optimiser. The chain's error on that object is coherent (share 0.70; a control with

independent per-layer subspaces, −0.002), but 200× too small and with the wrong sign of slope, so the preregistered predictions fail. Cross-layer alignment of the update is what makes the error coherent, which is Artyom_Mazur's Proposition 1 (18249) from the other side; training sets the magnitude and sign by some route we have not found.

What we take from it, for now

Training leaves a low-rank update sitting on top of a mostly intact random skeleton, and the two interact: a small slice of the skeleton, the part that overlaps the update, becomes load-bearing, and how much depends on how far the update has grown relative to what decay has shrunk. The random-net chain sees none of that structure in energy and all of it in its error, which turns from isotropic noise into a same-signed shortfall along the mean, the one direction the survival law says the chain cannot forget. The chain is also fragile off its distribution (non-finite on every 2-class net, on depth-24 nets inside the budget, and at lr 1e-3; 0 of 14 inits failed). One chain, one input law, mostly Gaussian-mixture tasks, some thin cells; the corpus is public so someone can run a different estimator on it.

Credits and tools

ARC (Wu, Lecomte, Winer, Robinson, Hilton, Christiano) for the framework and reference code; 504aldo (18218) for the public line our port was checked against; megan_park (18282), dogus_ozel (18281) and AndreasHad04 (18292) for the survival law; Artyom_Mazur (18249) and il.miscusi (18248) for Proposition 1 and the scale mode; michael_anderson7 (18272) for crediting the Phase 1 cancellation account; keenanpepper for the public corpus; AIcrowd and mohanty for the rulings. Claude Code (Anthropic; Claude Opus and Fable 5) wrote and ran the code and drafted this text under James Henry's direction; James set the direction, chose the pivot to trained weights, rejected the accuracy-only extraction metric (which led to the fidelity test), and reviewed every claim. Pre-registration files were written by the model and committed before the runs.